



Discrete Optimization

A memetic algorithm for the orienteering problem with hotel selection

A. Divsalar^{a,b,*}, P. Vansteenwegen^a, K. Sörensen^c, D. Cattrysse^a^a KU Leuven, Centre for Industrial Management/Traffic and Infrastructure, Celestijnenlaan 300A, 3001 Leuven, Belgium^b Faculty of Mechanical Engineering, Babol University of Technology, Babol, Mazandaran, Iran^c University of Antwerp, Prinsstraat 13, 2000 Antwerp, Belgium

ARTICLE INFO

Article history:

Received 13 June 2013

Accepted 3 January 2014

Available online 11 January 2014

Keywords:

Orienteering problem

Memetic algorithm

Population management

Intermediate facilities

Hotel selection

ABSTRACT

In this paper, a memetic algorithm is developed to solve the orienteering problem with hotel selection (OPHS). The algorithm consists of two levels: a genetic component mainly focuses on finding a good sequence of intermediate hotels, whereas six local search moves embedded in a variable neighborhood structure deal with the selection and sequencing of vertices between the hotels. A set of 176 new and larger benchmark instances of OPHS are created based on optimal solutions of regular orienteering problems. Our algorithm is applied on these new instances as well as on 224 benchmark instances from the literature. The results are compared with the known optimal solutions and with the only other existing algorithm for this problem. The results clearly show that our memetic algorithm outperforms the existing algorithm in terms of solution quality and computational time. A sensitivity analysis shows the significant impact of the number of possible sequences of hotels on the difficulty of an OPHS instance.

© 2014 Elsevier B.V. All rights reserved.

1. Introduction

The Orienteering Problem with Hotel Selection (OPHS) is a recently introduced variant of the orienteering problem (Divsalar, Vansteenwegen, & Cattrysse, 2013). In this problem, on a given complete graph $G=(V,E)$, a set of $h+n$ vertices is given. $V=H \cup N$ is composed of two subsets including h hotels, represented by numbers from 0 to $h-1$, and n points of interests (POI), represented by numbers from h to $h+n-1$. Hotel 0 and hotel 1 are used as the initial and the final depot respectively; the other hotels are called “extra hotels”. Each POI is assigned a score S_i . The symmetric travel time needed between each pair of vertices is given by t_{ij} . These travel times can be based on Euclidean distances or a travel time matrix. An ordered set of POIs with a specific start and end hotel is called a trip. The length of each trip $d=1, \dots, D$ is limited by a given time budget T_d . Each of the D connected trips starts and ends in one of the h hotels. The ordered set of trips that starts in the initial depot and ends in the final depot is called a tour. Both the initial and final depot can also be used as an intermediate hotel during the tour. The objective is to find a given number of connected trips visiting each POI at most once and maximizing the sum of collected scores.

Some examples of the large number of potential applications of the OPHS are explained in Divsalar et al. (2013): a submarine

performing a surveillance activity composed of consecutive missions, the design of a multi-day tourist trip through an attractive region, or a traveling salesperson who needs to select which of his possible clients to visit during his multiple day tour and also needs to choose the most appropriate hotels to stay the night.

We start this paper with an extensive literature survey on the OPHS and closely related problems. In Section 3 our heuristic algorithm is explained. Benchmark instances, experimental tests and a sensitivity analysis of the problem instance parameters are presented and discussed in Section 4 and the paper is concluded in Section 5.

2. Literature review

In the regular Orienteering Problem (OP), there are N vertices available, each with a score S_i , and the goal is to find a tour that visits some of these vertices, respects the available time limitation and maximizes the total collected score. The OP is also known as the selective traveling salesperson problem (Gendreau, Laporte, & Semet, 1998) and was introduced by (Tsiligirides, 1984). Since then, it has been considered in the literature for a number of applications such as home fuel delivery (Golden, Levy, & Vohra, 1987) and mobile tourist guides (Schilde, Doerner, Hartl, & Kiechle, 2009; Vansteenwegen, Souffriau, Vanden Berghe, & Van Oudheusden, 2011). Many (meta)heuristics and exact methods have been implemented to solve this problem. A recent survey on the OP and its variants, describing solution techniques, benchmark instances and applications can be found in (Vansteenwegen, Souffriau, & Van Oudheusden, 2011).

* Corresponding author at: KU Leuven, Faculty of Engineering, Department of Mechanical Engineering, Center for Industrial Management/Traffic & Infrastructure, Celestijnenlaan 300A Box 2422, BE-3001 Heverlee, Belgium. Tel.: +32 16 37 27 78.

E-mail addresses: ali.divsalar@cib.kuleuven.be, alidivsalar@gmail.com (A. Divsalar).

The hotel selection variant of the OP, studied in this paper, is closely related to a variant of the Vehicle Routing Problem (VRP) with so-called Intermediate Facilities (IF). To the best of our knowledge, (Angelelli & Speranza, 2002a) were the first to introduce intermediate facilities in a (node routing) VRP. The most important differences between their problem, the periodic VRP with Intermediate Facilities (PVRP-IF) and the OPHS are that in the OPHS each trip length is limited by a time budget and in the PVRP-IF the only time budget is the work shift applied for the whole tour and not per trip. Instead, a capacity constraint on the vehicle limits the number of customers that can be visited in each trip, while the OPHS has no capacity constraint. The tour time constraint and the fact that the number of trips is not fixed in the PVRP-IF makes it more complex to combine the trips in a tour, compared to the OPHS. Furthermore, in the PVRP-IF an empty vehicle leaves the depot and goes directly to an intermediate facility to load the goods. As a result, no customer can be visited on the way from the depot to the first facility. They propose a tabu search (TS) algorithm to solve this problem. In (Angelelli & Speranza, 2002b), the authors modify their PVRP-IF model and present a general model for waste-collection problems.

Crevier, Cordeau, and Laporte (2007) also address intermediate facilities in node routing problems with deliveries. They introduce an extension of the multi-depot VRP (MDVRP) in which vehicles may be replenished at intermediate facilities along their route. The Multi-Depot Vehicle Routing Problem with Inter-depot Routes (MDVRPI) deals with m vehicles of capacity Q . In contrast to the PVRP-IF, the MDVRPI is not a periodic problem. Moreover, a different terminology, new applications and a different solution strategy are introduced. The set of all routes assigned to a vehicle is called “rotation” which corresponds to the term “tour” in the OPHS. Based on this comparison, each route for each vehicle in the MDVRPI corresponds to a “trip” in the OPHS. There is a time limitation for a rotation of a vehicle in the MDVRPI. A rotation may be composed of “single-depot” and “inter-depot” routes which indicate if the starting and ending depot of a route are the same or different, respectively. Here again the problem differs from the OPHS in the sense that there is no explicit limitation on the length of each trip, only an implicit limitation based on the capacity of the vehicle. To solve the randomly generated instances of this problem, the authors propose a heuristic combining the adaptive memory principle, a tabu search method for the solution of sub problems, and integer programming. They apply their algorithm to a grocery distribution problem.

Tarantilis, Zachariadis, and Kiranoudis (2008) define the VRP with intermediate replenishment facilities (VRPIRF). In the VRPIRF, the goal is to determine optimal routes for a fleet of vehicles that can renew their capacity at intermediate replenishment stations. The VRPIRF is a special case of the MDVRPI in which one of the depots is called the central depot and the others being intermediate replenishment depots. The vehicles start and end their trips at the central depot. The part of the route of a vehicle between two consecutive depots is called a “route segment” and the total demand of the customers visited in each route segment is limited by the capacity of the vehicle. The duration of the whole route for a vehicle cannot exceed a given upper bound. Beside the non-periodic characteristic of the VRPIRF, the problem is very similar to the PVRP-IF except for a small difference: in the PVRP-IF the vehicles leave the central depot empty and immediately go to an intermediate facility and also have to visit an IF immediately before going back to the main depot at the end of their tour. This is not the case in the VRPIRF. Tarantilis et al. (2008) suggest a three-step algorithm to solve the VRPIRF. Their algorithm consists of a cost-saving construction heuristic to create the initial solution, improving it using a tabu search within a variable neighborhood search methodology, and applying guided local search to eliminate low-quality features from the final solution. They apply their

algorithm to benchmark instances in the literature as well as to new classes of the VRPIRF benchmark instances. They also mention some applications such as waste-collection when waste disposal is done to multiple dump sites, winter road maintenance when it is done by spreading chemicals and abrasives, and snow plowing.

Kim, Kim, and Sahoo (2006) present algorithms for a commercial waste collection VRP with time windows with consideration of multiple site locations and drivers' lunch break. They characterize this problem as a VRP with time windows and intermediate facilities (VRPTW-IF). In this specific waste collection application, a vehicle that is full needs to go to the closest available disposal facility and this may happen several times per day. There are two capacity constraints: one for the vehicle and another one for the route. The route capacity includes maximum number of stops and lifts, and maximum volume and weight that a driver can handle per day. Kim et al. (2006) take the minimization of route compactness and the workload balance into account besides the two common objective functions of VRPs: minimizing the number of vehicles and the travel time. These authors also state that considering disposal trips is not a simple task. An extended insertion algorithm of Solomon (Solomon, 1987) to consider multiple disposal trips and drivers' break, and a clustering-based waste collection VRPTW algorithm are proposed.

Kek, Cheu, and Meng (2008) introduce two new distance-constrained capacitated VRPs (DCVRP) and show the potential benefit of assigning start and end depots in a flexible way. The DCVRP_Fix is their first problem in which both vehicle capacity and maximum distance constraints are imposed as well as additional service and travel time constraints. It also considers the minimization of vehicle utilization and is applicable to both symmetric and asymmetric problems. In their second problem, the DCVRP_Flex, which is a relaxation of the first variant, the vehicles are allowed to start and end their tour at different depots and to visit any depot for reloading. In the definition of this problem, each vehicle route can pass through any number of depots any number of times as long as range constraints on the total length of any vehicle route are met. Their problem is structurally very similar to the MDVRPI. The main difference is that in the DCVRP_Flex a distance-constrained capacitated version of the VRP is considered to have intermediate facilities available. They solved both the DCVRP_Fix and the DCVRP_Flex problems to optimality using four case studies in Singapore and show the positive benefits of flexible assignment in practice. Comparing the definitions, one can find the similarities between the second version of the problem introduced by Kek et al. (2008) and the OPHS in the sense that in both problems the initial and the final depot are allowed to act as an intermediate depot as well. This is also the case for the MDVRPI. The difference is that in the OPHS the constraint of having a fixed start and end depot should still be satisfied while in the other mentioned problems (the MDVRPI and the DCVRP_Flex) it is relaxed. Moreover, the main difference is still the same: in the DCVRP_Flex, there is no time budget between each two consecutive depots in the route.

Time windows, driver breaks, and multiple disposal facilities in a waste collection VRP are also considered in Benjamin and Beasley (2010). They propose two metaheuristic algorithms using tabu search and variable neighborhood search as well as an algorithm based on variable neighborhood tabu search where the tabu search is used to search the neighborhood. As these authors mention, their problem is a collection problem with disposal facilities and it differs from the delivery problem with replenishment facilities in the sense that, in the collection problem, a vehicle visits a disposal facility to empty itself immediately prior to returning to the depot.

Since the OPHS is a node routing problem, arc routing problems with intermediate facilities (Ghiani & Federico, 2001; Ghiani, Guerriero, Laporte, & Musmanno, 2004; Ghiani, Laganà, Laporte, & Mari, 2008) are not discussed in detail here.

In conclusion, although there are some papers that consider intermediate facilities in node (and arc) routing problems with a cost (or time) minimizing objective function, there does not exist any research with intermediate facilities that considers maximizing the total collected score while the available time is not enough to visit all the customers. Beside this main difference none of the introduced problems imposes a time constraint on each trip length. Moreover, the number of connected trips in the tour of the OPHS is a given parameter of the problem which makes it different from all related problems mentioned above.

Another related problem to the OPHS is the traveling salesperson problem with hotel selection (TSPHS) which is introduced by Vans-teenwegen, Souffriau, and Sörensen (2012). In the TSPHS, the traveling salesman should select a hotel every night to rest and has to continue his tour the next day visiting the other customers from the same hotel. As he has to visit all his customers, the number of days is not predefined, but should be minimized as the primary goal in its objective function. The secondary goal is to minimize the total travel length. The authors develop a heuristic that uses two initialization methods and an improvement phase. The improvement phase is composed of seven local search moves and is specifically designed for their problem. Another recently published paper on the TSPHS (Castro, Sörensen, Vansteenwegen, & Goos, 2013) introduces a new mathematical formulation for the problem and designs a metaheuristic approach to solve it. Their suggested solution framework is a memetic algorithm with an embedded tabu search. For a detailed comparison between the TSPHS and the OPHS, we refer to the literature review presented in Divsalar et al. (2013).

In this final part of the literature review, we will briefly discuss the solution method for the OPHS described by Divsalar et al. (2013). In Divsalar et al. (2013), a skewed variable neighborhood search (SVNS) is developed to solve the OPHS. In the initialization phase of their SVNS, all the feasible sequences of hotels are created. Moreover, an orienteering problem is heuristically solved between each pair of hotels and using these results, the list of feasible sequences is sorted. Considering this list, the initial solution is constructed and some of the feasible sequences of hotels are selected and used in the improvement phase. The local search part of the SVNS is composed of nine local search moves as well as a shaking phase. In the shaking phase the hotel sequences are updated and the visited POIs between the hotels are also shaken. They also developed some OPHS benchmark instances with known optimal solution.

In the current paper, the local search part of the algorithm is a variable neighborhood descent (VND) in which less local search moves are used compared to the SVNS and the applied moves like Insert and Move-Best are implemented in a different way than in the SVNS. The (new) implementation of all moves is explained in Section 3.3. Moreover, the way that the two methods deal with the sequences of hotels is totally different. In the SVNS all the feasible sequences of hotels are created in advance which makes it inefficient for instances with a very large number of possible sequences. In the MA, this duty is done by the genetic operators of the algorithm which try to find a better sequence of hotels each time by combining the previously created sequences.

An overview of all mentioned literature is presented in Table 1.

Our choice for a memetic algorithm (MA) to tackle the OPHS is motivated by presenting some examples of successful implementations of an MA in solving the same class of problems. In the next section we will explain the suitable structure of an MA for solving the OPHS.

One can find a detailed survey of employing evolutionary algorithms for different variants of VRPs in Potvin (2009). MAs were first introduced by Moscato (1999) and have been successfully applied for VRPs by Prins (2004). Bouly, Dang, and Moukrim (2010) were the first to propose an MA for a variant of the OP, namely

the Team Orienteering Problem (TOP). Their MA uses an Optimal Split procedure (Beasley, 1983) specially developed for the TOP. Their Optimal Split procedure is later improved by Dang, Guibadj, and Moukrim (2013). In recent applications of MAs to VRPs, Mendoza, Castanier, Guéret, Medaglia, and Velasco (2010) propose an MA for the multi-compartment VRP with stochastic demands and Nguveu, Prins, and Wolfler Calvo (2010) develop an MA for the cumulative capacitated VRP. In the former, at each generation of a new population, crossover, mutation and local search operators are independently applied with a certain amount of probability for each. In the latter, at each iteration, two parents are selected by a binary tournament and a crossover is applied to generate eight children. Then the best child is selected using a kind of roulette wheel selection. Local search is applied to the best child and replaces a randomly selected individual among the 50% worst individuals of the current population.

3. A memetic algorithm for the OPHS

The structure of the OPHS, and the fact that it is not possible to predict which selection of hotels will result in a good solution, automatically leads to a two-level solution strategy. In this way, one level focuses on the hotel selection and the other part concentrates on the selection of POIs. A memetic algorithm (MA) corresponds naturally to this two-level solution strategy.

An MA is an evolutionary algorithm hybridized with local search techniques. The evolutionary algorithm that we use in our MA is a genetic algorithm (GA) focused on the (re)combination of different sequences of hotels (intermediate facilities). The local search technique is a variable neighborhood descent (VND) with six local search moves. The local search part focuses on finding good sequences of POIs between each pair of hotels in the tour.

For the solution encoding, we use D lists to represent the D trips. Each list starts and ends in a hotel. The start hotel of the first trip is the initial depot which is represented by 0 and the end hotel of the last trip is the final depot which is represented by 1. The POIs are mentioned by numbers between the start and end hotel of each trip based on their sequence of visit in the trip. Fig. 1 displays an example of the solution representation in the MA.

In the GA operators only the sequence of hotels is considered. Therefore, each GA chromosome only contains the sequence of visited hotels in the solution. In the example displayed in Fig. 1, the GA chromosome is also shown. The solution quality corresponding to this chromosome is the sum of the scores associated to the POIs visited in the solution.

Algorithm 1 shows the general structure of our algorithm. A detailed explanation of all elements of the algorithm, as well as of the local search moves, is given below.

Algorithm 1. General structure of the MA

```

1:  Initialization(Algorithm 1a)
2:  Main-loop:
3:      IterN ← 1
4:      While (IterN ≤ MaxIteration) Do
5:          |   Populate the Pool (Algorithm 1b)
6:          |   Sort the Pool according to solution quality
7:          |   Save the Best – Found – Solution
8:          |   Population management (Algorithm 1c)
9:          |   IterN ← IterN + 1
10:         End
11:  Output: “Best – Found – Solution”

```

The proposed MA is composed of two main parts: *Initialization* and *Main-loop*. In the *Initialization* part, a preprocessing step is performed and a potential score for each pair of hotels is determined.

Table 1
An overview of vehicle routing problems with intermediate facilities.

#	Reference	Problem Name	Problem specification	Objective function	Constraints	Solution Method	Case study	Instances		
								# Vertices	# IFs (depots/hotels)	# Paths (vehicles)
1	Angelelli and Speranza (2002a, (2002b)	PVRP-IF	VRP <i>m</i> Vehicles One depot Intermediate Facilities Time horizon	Min Total length	Vehicle capacity Work shift of the vehicle	Tabu Search	Waste collection	10–150	1–4	1–5
2	Kim et al. (2006)	VRPTW-IF	VRP Unlimited # vehicles One depot Intermediate facilities Time windows Drivers' lunch break	Min # vehicles Min travel time Max route compactness Balance workload	Vehicle capacity Route capacity Routing time per vehicle	Simulated Annealing Clustering	Waste collection	102–2092	1–19	3–17
3	Crevier et al. (2007)	MDVRPI	VRP <i>m</i> Vehicles	Min total travel duration	Vehicle capacity Total duration of a vehicle tour	Tabu Search + Mixed Integer Programming	Grocery distribution	48–216	3–6	4–6
4	Tarantilis et al. (2008)	VRPIRF	Intermediate depots Special case of MDVRPI <i>m</i> Vehicles One depot Intermediate facilities	Min total travel time	Vehicle capacity At most one route per vehicle Route duration constraint	Tabu Search within Variable Neighborhood Search + Guided Local Search	–	50–175	3–8	4–6
5	Kek et al. (2008)	DCVRP_Flex	VRP <i>m</i> vehicles Distance constrained Flexible trips	Min cost delivery route	Vehicle capacity Vehicle range: total length of any vehicle route	Mixed Integer Programming	Waste collection	2–8	1	1–2
6	Benjamin and Beasley (2010)	VRPTW-IF	VRP Unlimited # vehicles One depot Disposal facilities Time windows Driver rest	Min total travel distance	Max # of customers for a driver Max amount of waste collection for a driver per day Vehicle capacity	Tabu Search + Variable Neighborhood Search	Waste collection	102–2092	1–19	3–17
7	Vansteenwegen et al. (2012)	TSPHS	TSP Single vehicle Intermediate facilities	Min required number of trips Min total travel distance	Time budget for each trip	Multi Start Heuristic	–	10–1002	3, 5, 10	1–14
8	Castro et al. (2013)	TSPHS	TSP Single vehicle Intermediate facilities	Min required number of trips Min total travel distance	Time budget for each trip	Tabu Search within Memetic Algorithm	–	10–1002	3, 5, 10	1–14
9	Divsalar et al. (2013)	OPHS	OP <i>m</i> Trips Single vehicle Intermediate facilities	Max total score	Time budget for each trip	Skewed Variable Neighborhood Search	–	32–102	1–12	2–5

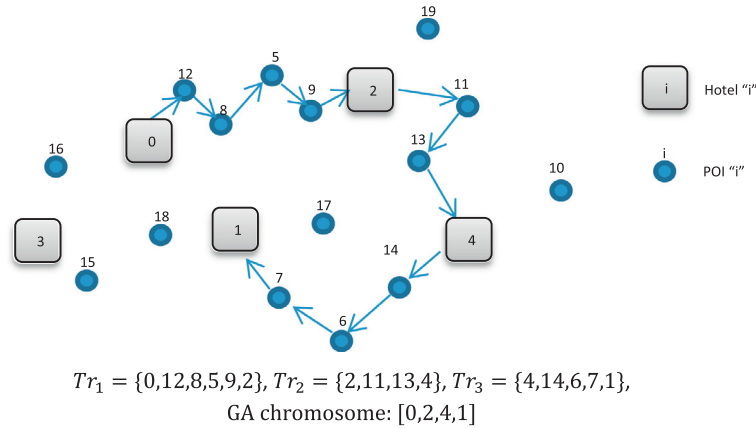


Fig. 1. Example of an OPHS solution representation.

Then, the initial population is created. The *Main-loop* part includes two crossover operators and one mutation operator that are used to populate the Pool of solutions (*Pool*). At the end of each iteration a new population is selected from the *Pool*. This process stops when a maximum number of iterations is reached.

3.1. Initialization

During the initialization, a potential score for every pairs of hotels is calculated first in a preprocessing step. Then, the initial population is generated. Algorithm 1a shows the initialization phase and in the following sections each part of it is explained in detail.

3.1.1. Matrices of pairs of hotels

An orienteering problem (OP) is heuristically solved between every possible pair of hotels for each trip. Note that each trip can have a different travel time limit and, as a result, each trip can have a different OP that needs to be solved. A very fast and straightforward algorithm which is called Greedy sub-OP heuristic by Divsalar et al. (2013) is used to solve these OPs independently from each other. This means that POIs can be visited more than once in different OPs. However, in this way, a potential score between each pair of hotels is calculated for each trip.

In this step, two three-dimensional matrices are created: one to save the solution, i.e., a sequence of POIs, between each pair of hotels (and for each trip) and the other one to save the score of this solution. From the matrix with scores, for each start hotel, $h_1 \in H$, and for each trip, $d \in \{1, \dots, D\}$, a list of possible end hotels, $h_2 \in H$, is derived. The possible end hotels are defined by two constraints: 1) $t_{h_1, h_2} \leq T_d$, and 2) $t_{h_2, 1} \leq \sum_{i=d+1}^D T_i$. In each list the possible end hotels are sorted based on the potential score of their corresponding trip. When later, during the *Main-loop* phase, some of the trip scores are updated; these sorted lists of hotels are updated as well.

Algorithm 1a. Initialization phase

```

1: Preprocessing:
2:   Create Matrices of pairs of hotels
3: Creating Initial Population:
4:    $k \leftarrow 1$ 
5:   While ( $k \leq \text{PopSize}$ ) Do
6:     |  $q_k \leftarrow$  Generate a feasible sequence of hotels
7:     | Apply Local Search on  $q_k$  to make a complete OPHS solution
8:     |  $k \leftarrow k + 1$ 
9:   End
10: Current Population  $\leftarrow$  Initial Population
11: Calculate and save the Best – Found – Solution

```

3.1.2. Creating initial population

Creating the initial population starts by composing sequences of hotels. Each time, the sequence starts in the initial hotel and the next hotels are selected one by one. The matrix of pairs of hotels includes all possible end hotels for each start hotel. This information is used here in making the sequence of hotels. The selection of an end hotel is not a pure random one, but the end hotels with higher scores have a larger probability of being selected (the selection probability is proportional to the score). Obviously, the sequences of hotels have to end in the final depot. If it happens in any trip that the selected hotels make it infeasible to end in the final depot, the algorithm replaces the last selected hotel by a different one based on the same procedure. If there is no more hotel left for the current trip, it goes back another step. After creating a feasible sequence of hotels, local search is applied on it to select POIs in all trips and to make a complete OPHS solution. The whole process is repeated to create enough solutions for the initial population.

3.2. Main-loop

3.2.1. Selection method

The roulette wheel selection method is used to select solutions for both *crossover operators* and the *mutation operator* (explained below). This method assigns to each solution in the population a probability for selection proportional to its solution quality. Let f_i be the solution quality value of solution P_i in the population P . This solution quality value corresponds to the total collected score of the solution. The probability of a solution to be selected is then calculated as $P_i = f_i / (\sum_{j=1}^n f_j)$ (Talbi, 2009).

3.2.2. Stopping condition

Setting an adequate stopping condition is crucial to balance solution quality and computation time of the algorithm. In our algorithm we simply use the maximum number of iterations (*MaxIteration*), i.e., the maximum number of new generations of the population. The exact value for *MaxIteration* will be discussed in Section 4.2 (line 4 of Algorithm 1).

3.2.3. Genetic operators

In each iteration of the main-loop, a Pool of solutions (*Pool*) is populated by the solutions of the current population as well as new solutions made by applying GA operators. First, the current population is transferred to the *Pool*. Then, two crossover operators and one mutation operator are used to create new solutions from the current population. These solutions are added to the *Pool*. The percentage of solutions selected from the current population to apply Crossovers I and II on, is set by CRI_R and CRII_R. Both

algorithm parameters are discussed in Section 4.2. Then, the mutation operator is applied on the current population, making more offspring to populate the *Pool* such that the *Pool* is twice the size of the current population. Algorithm 1b displays the whole process of populating the *Pool*. At the end of each iteration, the new population is selected from the *Pool*.

Algorithm 1b. Populating the *Pool*

```

1: Add the Current Population to the Pool
2:  $CRI_C \leftarrow 1$ 
3: While ( $CRI_C \leq CRI\_R \times PopSize$ ) Do
4:   | Select solutions  $P_1$  and  $P_2$  from the Current Population (Roulette Wheel)
5:   |  $O_1$  and  $O_2 \leftarrow$  Apply Crossover I on  $P_1$  and  $P_2$ 
6:   | Apply Local Search on  $O_1$  and  $O_2$ 
7:   | Add  $O_1$  and  $O_2$  to the Pool
8:   |  $CRI_C \leftarrow CRI_C + 1$ 
9: End
10:  $CRII_C \leftarrow 1$ 
11: While ( $CRII_C \leq CRII\_R \times PopSize$ ) Do
12:   | Select solutions  $P'_1$  and  $P'_2$  from the Current Population (Roulette Wheel)
13:   |  $O'_1$  and  $O'_2 \leftarrow$  Apply Crossover II on  $P'_1$  and  $P'_2$ 
14:   | Apply Local Search on  $O'_1$  and  $O'_2$ 
15:   | Add  $O'_1$  and  $O'_2$  to the Pool
16:   |  $CRII_C \leftarrow CRII_C + 1$ 
17: End
18:  $Mut_C \leftarrow 1$ 
19: While ( $Mut_C \leq (PopSize - (CRI\_R + CRII\_R) \times PopSize)$ ) Do
20:   | Select solution  $P''_1$  from the Current Population (Roulette Wheel)
21:   |  $O''_1 \leftarrow$  Apply Mutation on  $P''_1$ 
22:   | Apply Local Search on  $O''_1$ 
23:   | Add  $O''_1$  to the Pool
24:   |  $Mut_C \leftarrow Mut_C + 1$ 
25: End

```

3.2.3.1. Crossovers. A crossover operator is applied to two solutions selected from the current population as parents, and creates two new solutions, called offspring solutions, that will be added to the *Pool*.

In both crossover operators, the sequence of hotels is handled in the same way: the hotels of the selected parent solutions are combined using a one-point crossover to create two new hotel sequences. This is done in the following way:

- One of the trips (trip d) of the first parent, P_1 , is randomly selected. This trip is called the pivot trip.
- The algorithm then checks if it is feasible for this pivot trip to start from the initial hotel of the selected trip in the first parent solution, $h_{1,d}^{P_1}$, and end in the final hotel of the selected trip in the second parent solution, $h_{2,d}^{P_2}$. This check, $t_{h_{1,d}^{P_1}, h_{2,d}^{P_2}} \leq T_d$, can be done efficiently by using the matrix of pairs of hotels.
- If the condition in (b) is true, the new sequence of hotels is formed by taking the hotel sequence of the first part of the first parent up to the initial hotel of the pivot trip, $h_{1,d}^{P_1}$, and combining it with the hotel sequence from the final hotel of the pivot trip in the second parent, $h_{2,d}^{P_2}$, to the final depot.
- If the condition in (b) is false, another trip is selected randomly.

The same strategy is also applied to the second parent to make the second offspring solution. The crossover operator is illustrated in Fig. 2. It should be mentioned that the crossover point to create the second offspring solution is not necessarily the same as the crossover point to create the first.

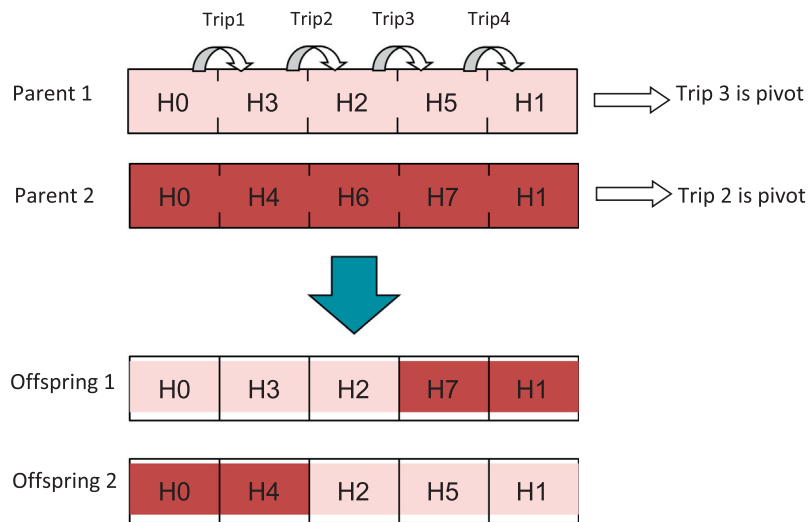


Fig. 2. Hotel sequences in both crossovers .

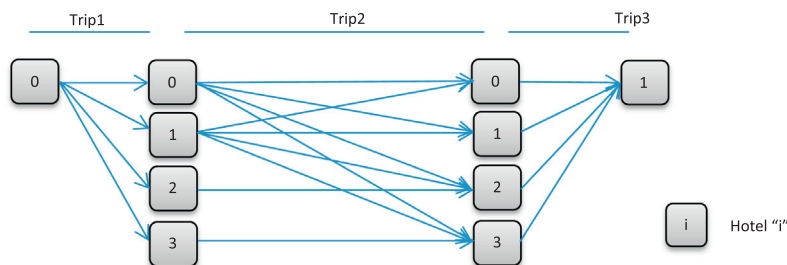


Fig. 3. Total Number of Feasible Sequences of hotels (TNFS).

The difference between the two crossover operators lies in how they continue from these two new hotel sequences.

3.2.3.1.1. Crossover I. This crossover operator implements a move specifically developed for the sequences of hotels in each offspring solution (called hotel-exchange). This move evaluates all feasible swaps of two hotels. Based on the scores saved in the matrix of pairs of hotels, the best swap is executed. In this step, for the trips that at least have one hotel changed, the heuristic solution saved in the matrix of pairs of hotels is used. Afterwards, the possible duplicate POIs are removed, first from the trips where at least one of the hotels was changed and then from the other trips in the solution. At the end, local search is applied to further improve the solution. So, in Crossover I, only for some trips the POIs in the parent solutions are used in creating the offspring.

3.2.3.1.2. Crossover II. In this operator, the POIs between the hotels in the parent solutions are used for the offspring solutions. For the pivot trip, the first hotel and all the POIs are copied from the first parent, and only the final hotel is derived from the second parent. If this makes the trip infeasible, the POIs with the least ratio of score over the saved time are removed one by one to make the trip feasible. In the end, local search is applied to further improve the solution.

3.2.3.2. Mutation. In the mutation operator, one solution is selected as a parent and mutated to create an offspring. To do this, one trip is randomly selected and the last hotel of the trip (equal to the first hotel of the next trip) is updated as described below. In some cases also the last hotel of the next trip need to be updated, but never any of the other hotels.

For each hotel and each trip a sorted list of possible end hotels is available from the matrix of pairs of hotels. The new end hotel for the selected trip (pivot trip) is selected from this list based on the first hotel of the trip. Starting from this new end hotel it is checked that by selecting this hotel the next trip will have a feasible pair of hotels (looking at the end hotel of the next trip). If this new hotel leads to an infeasible sequence for the next trip, the last hotel in the next trip is also considered for updating using the same procedure. If by selecting an end hotel for the pivot trip, it is not possible to find a feasible end hotel for the next trip, the next possible end hotel is considered for the pivot trip. In the few cases that no feasible combination of end hotel for the pivot trip and end hotel for the next trip can be determined, the mutation operator will not modify the parent.

After updating the hotel(s), the two or three affected trips are filled using the solutions saved in the matrix of pairs of hotels. The other trips stay the same in the beginning. To make the solution feasible, first the duplicated POIs are removed from the unchanged trips and then from the affected trips. Finally, local search is applied to further improve the solution.

At the end of each mutation iteration, the matrix of pairs of hotels is updated: the assigned scores for the pairs of hotels which are new in the offspring compared to the parent are updated in the matrix and the sorted list of hotels is updated as well. To ensure the variety of different possible hotel sequences, a tabu list is used to store the recently selected hotels in each trip. The tabu list is updated each time the mutation is performed. The new selected end hotel of each trip (one or two updated hotels), is made tabu for a number of iterations. The number of iterations of mutation for which the same hotel is not selected for the same trip is *TabuSize*, a parameter of the algorithm. This will be discussed in Section 4.2.

Algorithm 1c. Population management

```

1: Empty the Current Generation
2:  $Ind_{c1} \leftarrow 1$ 
3: While ( $Ind_{c1} \leq BestSel\_R \times 2 \times PopSize$ ) Do
4:   |  $i \leftarrow Ind_{c1}$ 
5:   | Select the  $i^{th}$  solution from the top of the Pool and
   | add it to the Current Population
6:   |  $Ind_{c1} \leftarrow Ind_{c1} + 1$ 
7: End
8:  $Ind_{c2} \leftarrow Ind_{c1}$ 
9: While ( $Ind_{c1} \leq 2 \times PopSize$  and  $Ind_{c2} \leq PopSize$ ) Do
10:  |  $i \leftarrow Ind_{c1}$ 
11:  | If ( $i^{th}$  solution,  $P^i$ , is different from the solutions
   | | in the Current Population) then
12:  | | Add  $P^i$  to the Current Population
13:  | |  $Ind_{c2} \leftarrow Ind_{c2} + 1$ 
14:  | End
15:  |  $Ind_{c1} \leftarrow Ind_{c1} + 1$ ;
16: End
17: If ( $Ind_{c1} < PopSize$ ) then
18:  |  $i \leftarrow 2 \times PopSize$ 
19:  | While ( $Ind_{c1} \leq PopSize$ ) Do
20:  | | Select the  $i^{th}$  solution from the bottom of the Pool
   | | and add it to the Current Population
21:  | |  $i \leftarrow i - 1$ 
22:  | |  $Ind_{c1} \leftarrow Ind_{c1} + 1$ 
23:  | End
24: End

```

3.2.4. Population management

In the next step, the next population is selected from the Pool that currently contains both the current population and the newly created offspring. This selection is done based on a strategy to manage the population considering both the quality and the diversity of the solutions contained in it. In this view, our algorithm belongs to the class of memetic algorithms with population management (MA|PM), introduced in (Sörensen & Sevaux, 2006) and extensively described and analyzed in (Boudia & Prins, 2009). Most population management strategies are based on an incremental genetic algorithm and involve a distance measure between solutions. Here we develop a generational genetic algorithm with a simpler and more flexible procedure to manage population diversity. The process is depicted in Algorithm 1c. In order to apply our population management strategy, all the solutions in the Pool are sorted based on their quality. Afterwards, the solutions with the best solution quality are selected to be in the next population which becomes the current population of the next iteration. The number of selected solutions in this step is equal to $BestSel_R \times 2 \times PopSize$ in which both *BestSel.R* and *PopSize* are parameters of the algorithm that will be discussed in Section 4. To fill the rest of the population, the same order is used, but only solutions with a sequence of hotels different from the already selected solutions are chosen. If the number of solutions with different sequences of hotels is not enough to fill the population, the solutions with the minimum solution quality are selected to fill the population. In this way, we try to keep enough diversity in the population.

3.3. Local search

Local search tries to improve the selection of POIs between the hotels. A variable neighborhood descent with six well known local search moves is implemented. An overview is given in Algorithm 2. Next, all moves are briefly explained.

Algorithm 2. Local search

```

1:  X: The incumbent solution of the Local Search
2:  Set of six neighborhood structures ( $N_k$ ):
3:  Insert, Move-Best, Two-Opt, Swap- Best, Extract-Insert, Extract2-Insert;
4:   $k \leftarrow 0$ 
5:  While ( $k < 6$ ) Do
6:    |  $X' \leftarrow$  Apply neighborhood structure  $N_k$  on  $X$ 
7:    | If  $X'$  is better than  $X$  then
8:    | |  $X \leftarrow X'$ 
9:    | |  $k \leftarrow 0$ 
10:   | Else
11:   | |  $k \leftarrow k + 1$ 
12:   | End
13: End

```

Insert: For each non-included POI, the position with minimum increase in trip length in any of the trips is determined. If this position leads to a feasible solution, the POI is inserted. Then, the next POI is considered until no more insertions are possible.

Move-Best: For every visited POI in every trip, it is checked that removing this POI from its current position and putting it in its best possible position within the whole tour is feasible and decreases the length of the tour. Only the feasible move with the highest decrease in total tour length is completed.

Two-Opt: For each trip, two visited POIs are determined for which the inversion of the order of vertices between them leads to the largest travel time decrease in the trip. Then, the order of visiting the POIs between them is reversed.

Swap-Best: This move exchanges two visited POIs between two different trips to reduce the total travel time. For each pair of trips ($d1$ and $d2$) two POIs (one from each trip) are swapped only if the length of at least one of the trips decreases and both trips remain feasible. In order to determine this, four travel time changes are calculated: the amount of time saved in trip $d1$ by excluding POI i ; the amount of time saved in trip $d2$ by excluding POI j ; the best position in trip $d1$ to include POI j , and the best position in trip $d2$ to include POI i (least time consuming positions). Only the move leading to the highest total time saving is performed.

Extract-Insert: This move starts from the first visited POI in each trip and checks if by excluding this vertex and inserting as many unvisited POIs as possible in the trip, the score can be increased. If that is the case, this exchange is applied and the algorithm goes to the next visited POI in the trip. The excluded POI is not considered again for insertion. Inserting the POIs is done by *Insert*, described above.

Extract2-Insert: This move is identical to the previous move, except for the fact that it always considers two consecutive visited POIs for exclusion.

4. Experiments and results

The benchmark instances are described briefly in Section 4.1 and the preliminary experiments used to set the parameters are discussed in Section 4.2. The results of the experiments are presented and explained in Section 4.3. An important observation is that the total number of feasible sequences of hotels (TNFS) is a key element determining the difficulty of an OPHS instance and is discussed in Section 4.4. Finally, in Section 4.5, the sensitivity of the results with respect to changes in parameters and parts of the algorithm is analyzed.

4.1. Implementation and instances

In Divsalar et al. (2013), it is explained how different sets of test instances are created with known optimal solutions. These instances are created based on the optimal solution of regular OP instances. In order to create OPHS instances with D trips, $D - 1$ hotels are added at certain POIs in the optimal path and some additional hotels are added on the place of randomly selected POIs from the OP instance. The key point is that these $D - 1$ hotels are added and the trip lengths are fixed in such a way that the optimal OPHS solution corresponds to the optimal OP solution. By solving some larger OP instances to optimality, thanks to a more powerful PC and a better coding, we were able to create more and larger instances of the OPHS with known optimal solutions in the same way. These new instances are also available at <http://www.mech.kuleuven.be/en/cib/op>. In order to evaluate the performance of the proposed algorithm, it is tested on all previously available benchmark instances as well as the newly created instances. The new set of instances are indicated in Table 2 by an * at the beginning of their name in the first column. The algorithm is coded in Visual C++ 2010. All computations were carried out on a personal computer Intel Core 2 with 3.00 GHz processor and 4.00 GB RAM.

4.2. Parameter tuning

In total there are six parameters involved in the proposed algorithm: *MaxIteration*, *PopSize*, *CRI_R*, *CRII_R*, *BestSel_R* and *TabuSize*. To set these parameters properly, a number of pilot experiments are performed. First, 28 instances are randomly chosen from different sets of instances. Then, for each of the parameters, three different values are determined. Afterwards, the algorithm is applied on the selected set for all 729 possible combinations of different values of parameters. In order to select the appropriate set of parameter values, we made a trade-off between quality of the results and the computation time. Based on these pilot experiments, the parameters are set as follows: *MaxIteration* = 100, *PopSize* = 30, *CRI_R* = 30%, *CRII_R* = 40%, *BestSel_R* = 25% and *TabuSize* = 33% of the number of hotels.

4.3. Results

Detailed results for each instance can be found in the tables presented in Appendix A. The results are compared with the known optimal solution and the results of the skewed variable neighborhood search algorithm (SVNS) published in Divsalar et al. (2013). A summary of the results on all presented instances in Appendix A, is displayed in Table 2. The presented sets of instances in this table are ordered based on the maximum total number of feasible sequences of hotels (TNFS) in each set. Our experiments show that TNFS is an important characteristic of an OPHS instance, determining its difficulty. This characteristic is investigated in detail in Section 4.4. In Table 2, the first column shows the set of instances. The name of each set of instances shows the number of hotels, excluding the initial and final depot, as well as the number of trips in each corresponding instance. The second column indicates the number of instances in each set. Then the maximum number of TNFS in each set is presented in the third column. Afterwards, the average gap, the maximal gap and the number of instances for which the optimal solution is found are displayed. Due to randomness in its structure, MA is applied on each instance three times. But since there is no randomness in the SVNS structure, only one result is

Table 2

The results of the experiments.

Name	Set of Instances		Average Gap			Max Gap			# Opt			Average CPU	
	# Instances	Max TNFS	MA		SVNS	MA		SVNS	MA		SVNS	MA	SVNS
			Avg	Best		Avg	Best		AVG	Best			
1 Hotel – 2 trips (Extra LS moves)	35	3	1.44	1.29	1.22	7.69	7.69	7.55	15	17	18	2.59	0.12
1 Hotel – 2 trips	35	3	3.32	2.82	1.22	13.91	13.91	7.55	11	14	18	1.84	0.12
2 Hotels – 3 trips	35	16	0.88	0.43	0.93	4.34	3.33	5.19	17	23	19	1.43	0.14
SET 4 (OPT)	5	25	0.76	0.76	0.76	3.82	3.82	3.82	4	4	4	1.17	0.24
5 Hotels – 3 trips	35	49	0.81	0.41	0.93	3.42	2.67	4.05	18	24	19	1.41	0.21
3 Hotels – 4 trips	35	125	0.60	0.46	0.92	4.89	4.67	5.00	21	23	18	1.09	0.41
6 Hotels – 4 trips	35	512	0.64	0.39	1.21	4.22	3.33	5.19	18	23	17	1.15	0.88
10 Hotels – 4 trips	22	1728	1.72	0.96	2.61	4.01	3.00	8.17	3	7	3	6.59	5.12
*12 Hotels – 4 trips	22	2744	1.78	1.24	2.73	5.16	4.55	6.08	5	6	2	6.50	5.01
*15 Hotels – 4 trips	22	4913	1.92	1.32	2.66	5.15	4.13	5.76	3	5	2	6.63	4.95
*10 Hotels – 5 trips	22	20736	1.99	1.30	4.15	6.40	4.94	9.93	3	6	4	5.43	4.26
12 Hotels – 5 trips	22	38416	1.98	1.43	3.58	5.15	4.61	8.68	4	6	4	5.44	4.21
*15 Hotels – 5 trips	22	83521	2.22	1.40	4.42	5.80	4.61	11.52	3	7	4	5.41	4.2
*10 Hotels – 6 trips	22	248832	2.30	1.24	6.27	7.70	6.87	12.77	4	9	4	4.66	3.83
*12 Hotels – 6 trips	22	537824	2.28	1.63	5.63	6.24	5.21	12.85	5	6	3	3.49	3.84
*15 Hotels – 6 trips	22	1419857	2.55	1.39	6.86	6.11	5.75	13.52	4	8	4	4.78	4.29
*15 Hotels – 8 trips	13	410338673	3.66	2.95	15.54	9.75	8.80	22.37	1	1	0	5.16	53.62
*15 Hotels – 10 trips	9	$1.185878765 \times 10^{11}$	5.03	3.78	–	9.25	9.16	–	1	2	0	5.04	–
All instances	400	1.18×10^{11}	1.82	1.24	3.98	13.91	13.91	22.37	125	174	125	3.59	4.08

shown for it. So, for each of these mentioned columns, three sub-columns show the average and the best results of three times applying MA and the result of applying SVNS, respectively. The gap here is the percentage difference between the obtained solution and the optimal solution, calculated this way: $100 \times \left(\frac{\text{Optimal result} - \text{MA result}}{\text{Optimal result}} \right) \%$. The last column, Average CPU, has two sub-columns and presents the average computational time spent in solving each instance set applying MA and SVNS. The presented computational time of applying the MA shows the average time of three runs on each instance. The (small) difference in computation times reported here for the SVNS compared to those reported in Divsalar et al. (2013) are due to a different computing environment. The times reported here allow a fair comparison of both techniques.

The set of instances with 1 extra hotel (in addition to the initial and final depot) and 2 trips is displayed two times with different results since a second version of the algorithm was applied on this set. This is explained later. The set of instances presented in the fourth row of the results in Table 2, corresponds to the so called SET 4 (Divsalar et al., 2013), but in Table 2 only the five instances with known optimal solution are used.

We start the discussion of the results in Table 2 with the exceptional results on the first set (second row of results). The special characteristic of these very small instances is the very small number of TNFS. Clearly, our MA does not outperform the SVNS on these very small instances (Appendix A, Table A.2). Actually, the whole structure of the MA, focusing on finding the best sequence of hotels, becomes useless when only three feasible sequences of hotels are possible. For these instances, the algorithm should focus only on solving the combination of OPs between the selected hotels. Actually, some extra experiments show that if we add four more local search moves to the Local Search part of our MA algorithm, hereby focusing more on solving the combination of OPs, the average results of our algorithm correspond to the SVNS results, but the MA is still slower (Appendix A, Table A.1). The results obtained by this second version of the algorithm are presented in the first results row of Table 2 and are not counted in the total row (last row) of Table 2.

In all other sets of instances, with the TNFS ranging from 16 to 1.18×10^{11} , for both average and best results of three runs of the MA, the average gap obtained by the MA is less than the average

gap obtained by the SVNS. Moreover, the results achieved by the MA are much better than the SVNS results for larger instances. This conclusion can be seen by comparing both average and maximal gaps.

When it comes to the computation time, the comparison shows a different trend. For smaller sets of instances, the SVNS is much faster than the MA. But when the number of TNFS increases, the difference between average computational times decreases and for the largest sets of instances with the largest number of TNFS, the MA is much faster than the SVNS. The results show that for instances with large number of TNFS, the SVNS is very time consuming, but the MA is able to solve these in a reasonable time.

4.4. Total number of feasible sequences of hotels (TNFS)

The maximal value of TNFS for any instance is equal to $(h)^{(D-1)}$. However, since there might be some infeasible sequences of hotels, we created and counted all feasible sequences of hotels using the initialization part of the SVNS algorithm. For larger instances, for which SVNS could not find all the feasible sequences due to memory shortage, the maximal value of TNFS is used.

Fig. 3 is an example of how TNFS is computed using an OPHS instance of 3 trips and 2 intermediate hotels ($h = 4$).

In this example the max value of $TNFS = (4)^{(3-1)} = 16$, but as some of the connections in trip 2 are not feasible (for example, trip 2 is too short to go from hotel 2 to hotel 3), so the actual TNFS for this example is 10.

To have a better insight in the significant impact of TNFS on the complexity of the problem and thus the performance of the MA and SVNS, Figs. 4 and 5 present the experimental results graphically. This also allows comparing the results of the two algorithms. In both figures, the horizontal axis shows the maximum number of TNFS in each set of instances. The label SVNS shows the results from the SVNS algorithm presented in Divsalar et al. (2013) and MA-AVG and MA-Best are the labels of the average and best results of three runs of the MA on each set of instances, respectively.

In Fig. 4, the vertical axis shows the average gap, in percentage, between the results of the algorithms and the optimal solution. The figure clearly shows that the average gap increases when the TNFS increases. Moreover, it can be seen that almost everywhere the average gap of the results obtained by the MA is less than the

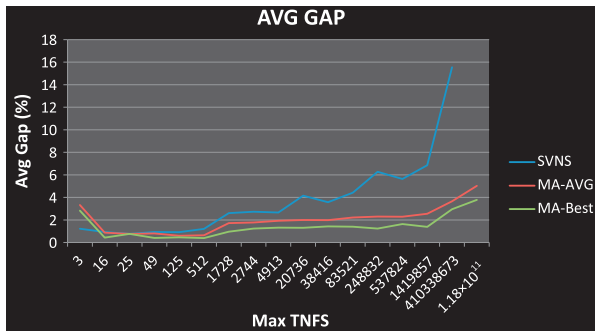


Fig. 4. Comparison of average gap in each set of instances.

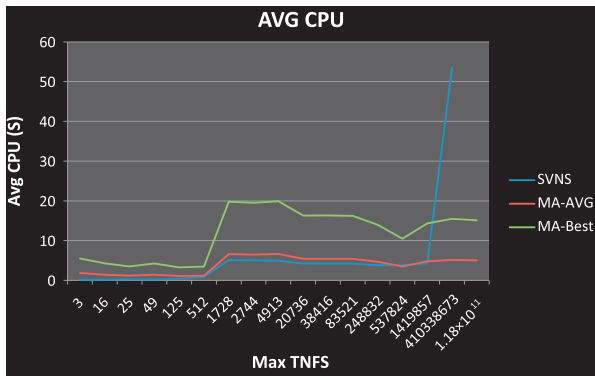


Fig. 5. Comparison of average computational time in each set of instances.

one from the SVNS and when *TNFS* increases, the distance between the SVNS and the MA results significantly increases.

In Fig. 5, a comparison is shown between the computational time of each of the two algorithms. Considering the Average CPU usage of the three runs of the MA, for smaller instances SVNS is faster. However, when *TNFS* is significantly increased, the MA is clearly much faster than the SVNS.

4.5. Parameter sensitivity

Some extra computational experiments, summarized in Table 3, show that small changes in the parameter values do not have a significant impact on the performance of the algorithm. In Table 3, the first column under the name of MA presents the values of the parameter used in the algorithm to obtain the presented results

Table 3
Sensitivity analysis I.

Parameters	MA			State 1			State 2		
	Parameter value	Average Gap (%)	Average CPU time	Parameter value	Average Gap (%)	Average CPU time	Parameter value	Average Gap (%)	Average CPU time
PopSize	30	3.66	5.16	20	3.98	3.46	40	3.45	6.87
MaxIteration	100	3.66	5.16	50	5.13	2.67	200	3.02	10.01
BestSel_R	25% of PopSize	3.66	5.16	10% of PopSize	3.06	5.11	33% of PopSize	3.8	5.06
CR1	30% of PopSize	3.66	5.26	20% of PopSize	3.36	4.97	40% of PopSize	3.48	5.54
CR2	40% of PopSize	3.66	5.16	30% of PopSize	3.18	5.3	50% of PopSize	3.46	5.05
TabuSize	33% of H	3.66	5.16	25% of H	3.46	5.11	50% of H	3.39	5.2

Table 4
Sensitivity analysis II.

State	Average gap (%)	Average CPU time
MA	3.66	5.16
Without CR1	5.07	4.35
Without CR2	4.84	5.63
Without Mutation	4.47	5.59

in Table 2. The second and third column under MA show the results of applying these parameter values on the set of instances with 15 extra hotels and 8 trips. The second and third group of three columns present the results of applying the MA with one slightly different parameter value on the same set of instances.

The only changes in parameter values that do have a significant impact on the performance of the algorithm are, obviously, *MaxIteration* and *PopSize*. By using higher values for them the quality of the results are increased but at the cost of higher computation time.

Some extra experiments also illustrate that both crossovers and the mutation are useful operators in our MA algorithm. To show this, the crossovers and the mutation are removed one by one and the impact on the selected set of instances is investigated. In the algorithm setting “Without Mutation”, the weights assigned to both crossovers are equal to 50% of the *PopSize*. The results from this analysis are presented in Table 4. Removing any of these parts significantly increases the average gap with the optimal solutions.

5. Conclusion

The OPHS is a challenging combinatorial optimization problem. Although “intermediate facilities” are considered in some variants of the VRP, the OPHS is studied in only one paper in the literature. A detailed overview of all related problems, also discussing the similarities and differences with the OPHS, is presented in this paper.

We create new and larger instances of the OPHS with up to 15 extra hotels and 10 trips. This size is realistic in the context of tourism applications. These instances are efficiently solved by our algorithm in a reasonable time. Moreover, it is clearly shown that the number of possible sequences of hotels is a critical characteristic of an OPHS instance to indicate its difficulty.

The proposed memetic algorithm is designed to deal with larger instances of the OPHS and it certainly outperforms the only other existing algorithm, SVNS, on these instances. The two-level structure of the MA is efficiently compatible with the two different sets of nodes in each instance: hotels and POIs. Therefore, the higher level uses the genetic algorithm property to make a diverse list of

different sequences of hotels. Moreover, the fact of creating a population of solutions considering different sequences of hotels guarantees the diversity of the population. The lower level of the MA is the POI level which uses local search moves to efficiently solve the combination of OPs between the selected hotels. The fact that both quality and diversity in the selection of individuals from the Pool introduces a simple and flexible form of population management and can be considered another advantage of the proposed memetic algorithm. The special design of mutation is another strong point of the algorithm. In this operator, different strategies are used to escape from local optima by perturbing the order of POIs even between unchanged hotels.

Extensions of the OPHS to other versions of the problem, including time windows, hotel scores, route scores or an OPHS with multiple vehicles, can be considered in future work. These proposed variants of the problem will help to model more realistic situations in tourism and other possible applications of the problem.

Appendix A. Detailed results of the experiments

Tables A.1–A.7, A.11 and A.18 present results on the benchmark instances of (Divsalar et al., 2013) and Tables A.8–A.10 and A.12–A.17 show the results on the new instances. In all displayed tables, the second column, *TNFS*, shows the total number of feasible sequences of hotels. The optimal value for each instance is displayed

in the third column of the tables. Due to randomness in our MA algorithm, it was run three times for each instance. Therefore, the worst, the average, and the best gaps with the optimal values are presented in columns 4–6 of each table $100 \times \left(\frac{\text{Optimal result} - \text{MA result}}{\text{Optimal result}} \right) \%$. Then, to be able to have an easy comparison, the results obtained by SVNS are presented in the next column. The computation times are displayed in two columns under the name of CPU. First one, MA-CPU, shows the average computation time of three runs of the MA algorithm and next to that the computation time of the SVNS algorithm is presented. For Table A.16, SVNS is only able to solve 7 out of 13 presented instances. For the other instances in Table A.16 as well as all the instances presented in Table A.17 no feasible solution is found by SVNS in 360 s of computation time per instance.

In Tables A.16 and A.17, the presented maximum number of *TNFS* is the theoretical maximum, which equals $H^{(D-1)}$, since for at least some of the instances the real feasible number of sequences of hotels was too large to calculate.

The presented results in Table A.18 only include the average results of applying MA three times on each instance. In this table, after the instance name and its *TNFS*, the best feasible (BF), upper bound (UB) and optimal value (OV) for the corresponding instances are presented. The way of creating these instances and the mentioned values are explained in Divsalar et al. (2013).

Table A.1

1 Extra hotel – 2 trips (when more LS moves are used).

1 Extra hotel 2 Trips			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Instances								
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	3	240	2.08	2.08	2.08	0.00	0.62	0.04
T1_70	3	260	5.77	5.77	5.77	0.00	0.66	0.04
T1_73	3	265	0.00	0.00	0.00	7.55	0.67	0.03
T1_75	3	270	0.00	0.00	0.00	0.00	0.60	0.03
T1_80	3	280	3.57	3.57	3.57	3.57	0.61	0.03
T1_85	3	285	0.00	0.00	0.00	1.75	0.50	0.04
T3_65	3	610	0.00	0.00	0.00	0.00	0.53	0.04
T3_75	3	670	0.00	0.00	0.00	2.99	0.57	0.03
T3_80	3	710	0.00	0.00	0.00	2.82	0.64	0.04
T3_85	3	740	4.05	4.05	4.05	0.00	0.64	0.03
T3_90	3	770	1.30	1.30	1.30	0.00	0.59	0.04
T3_95	3	790	0.00	0.00	0.00	1.27	0.47	0.03
T3_100	3	800	0.00	3.33	5.00	3.75	0.52	0.03
T3_105	3	800	0.00	0.00	0.00	0.00	0.58	0.03
64_45	3	816	0.74	0.98	1.47	0.00	6.47	0.19
64_50	3	900	2.67	2.67	2.67	2.67	6.08	0.26
64_55	3	984	0.61	1.02	1.22	1.22	6.92	0.24
64_60	3	1062	1.69	1.69	1.69	1.13	9.84	0.27
64_65	3	1116	0.00	0.36	1.08	0.00	5.77	0.29
64_70	3	1188	1.52	1.68	2.02	1.52	6.22	0.24
64_75	3	1236	1.46	1.46	1.46	0.00	6.05	0.29
64_80	3	1284	0.93	0.93	0.93	0.00	6.63	0.24
66_40	3	575	0.87	0.87	0.87	0.87	1.39	0.09
66_45	3	650	7.69	7.69	7.69	0.77	2.13	0.11
66_50	3	730	4.11	4.11	4.11	2.05	3.10	0.12
66_55	3	825	2.42	2.42	2.42	2.42	3.95	0.16
66_60	3	915	2.73	2.73	2.73	6.01	2.85	0.20
66_125	3	1670	0.90	1.50	1.80	0.30	4.18	0.22
66_130	3	1680	0.00	0.00	0.00	0.00	4.27	0.24
100_30	2	173	0.00	0.00	0.00	0.00	0.77	0.03
100_35	1	241	0.00	0.00	0.00	0.00	1.06	0.06
100_40	2	299	0.00	0.00	0.00	0.00	1.74	0.14
100_45	3	367	0.00	0.00	0.00	0.00	2.07	0.12
102_50	3	181	0.00	0.00	0.00	0.00	0.36	0.02
102_60	3	243	0.00	0.00	0.00	0.00	0.67	0.05
Average	2.89	–	1.29	1.44	1.54	1.22	2.59	0.12
Max	3	–	7.69	7.69	7.69	7.55	9.84	0.29

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.2

1 Extra hotel – 2 trips.

1 Extra hotel 2 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	3	240	2.08	2.08	2.08	0.00	0.40	0.04
T1_70	3	260	5.77	5.77	5.77	0.00	0.46	0.04
T1_73	3	265	0.00	0.00	0.00	7.55	0.43	0.03
T1_75	3	270	0.00	0.00	0.00	0.00	0.38	0.03
T1_80	3	280	3.57	3.57	3.57	3.57	0.42	0.03
T1_85	3	285	0.00	0.00	0.00	1.75	0.37	0.04
T3_65	3	610	0.00	0.00	0.00	0.00	0.42	0.04
T3_75	3	670	0.00	0.00	0.00	2.99	0.45	0.03
T3_80	3	710	0.00	0.94	2.82	2.82	0.46	0.04
T3_85	3	740	4.05	4.05	4.05	0.00	0.34	0.03
T3_90	3	770	0.00	0.00	0.00	0.00	0.37	0.04
T3_95	3	790	0.00	0.00	0.00	1.27	0.37	0.03
T3_100	3	800	6.25	6.25	6.25	3.75	0.41	0.03
T3_105	3	800	0.00	0.42	1.25	0.00	0.32	0.03
64_45	3	816	2.21	9.56	13.24	0.00	3.48	0.19
64_50	3	900	3.33	3.33	3.33	2.67	3.01	0.26
64_55	3	984	1.22	2.44	3.05	1.22	3.60	0.24
64_60	3	1062	1.13	3.58	8.47	1.13	5.44	0.27
64_65	3	1116	0.00	0.72	2.15	0.00	5.96	0.29
64_70	3	1188	2.02	2.53	3.03	1.52	4.60	0.24
64_75	3	1236	1.94	2.10	2.43	0.00	4.17	0.29
64_80	3	1284	0.93	2.18	4.67	0.00	4.71	0.24
66_40	3	575	13.91	13.91	13.91	0.87	1.31	0.09
66_45	3	650	8.46	8.46	8.46	0.77	1.29	0.11
66_50	3	730	4.11	4.11	4.11	2.05	2.55	0.12
66_55	3	825	2.42	4.04	4.85	2.42	1.36	0.16
66_60	3	915	9.29	10.02	11.48	6.01	2.13	0.20
66_125	3	1670	1.80	1.80	1.80	0.30	3.93	0.22
66_130	3	1680	0.00	0.00	0.00	0.00	3.88	0.24
100_30	2	173	5.78	5.78	5.78	0.00	0.40	0.03
100_35	1	241	0.00	0.00	0.00	0.00	0.78	0.06
100_40	2	299	0.00	0.00	0.00	0.00	0.68	0.14
100_45	3	367	0.00	0.00	0.00	0.00	1.28	0.12
102_50	3	181	11.05	11.05	11.05	0.00	0.25	0.02
102_60	3	243	7.41	7.41	7.41	0.00	0.51	0.05
Average	2.89	–	2.82	3.32	3.86	1.22	1.74	0.12
Max	3	–	13.91	13.91	13.91	7.55	5.96	0.29

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.3

2 Extra hotels – 3 trips.

2 Extra hotels 3 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	16	240	0.00	0.00	0.00	0.00	0.39	0.05
T1_70	16	260	0.00	0.00	0.00	0.00	0.41	0.04
T1_73	16	265	0.00	0.00	0.00	0.00	0.45	0.04
T1_75	16	270	0.00	0.00	0.00	0.00	0.36	0.07
T1_80	16	280	0.00	0.00	0.00	0.00	0.39	0.04
T1_85	16	285	0.00	0.00	0.00	0.00	0.46	0.05
T3_65	16	610	0.00	1.09	3.28	0.00	0.39	0.04
T3_75	16	670	0.00	0.00	0.00	2.99	0.48	0.05
T3_80	16	710	0.00	0.00	0.00	0.00	0.44	0.05
T3_85	16	740	0.00	2.70	4.05	0.00	0.43	0.07
T3_90	16	770	0.00	1.30	3.90	5.19	0.38	0.05
T3_95	16	790	0.00	0.00	0.00	0.00	0.49	0.06
T3_100	16	800	0.00	0.00	0.00	0.00	0.40	0.05
T3_105	16	800	0.00	0.83	1.25	1.25	0.43	0.05
64_45	16	816	1.47	1.47	1.47	0.00	1.55	0.18
64_50	16	900	3.33	3.33	3.33	2.67	1.98	0.25
64_55	16	984	1.22	2.85	4.88	3.05	2.40	0.26
64_60	16	1062	0.00	0.00	0.00	2.26	2.64	0.44
64_65	16	1116	0.00	0.18	0.54	2.15	3.71	0.33
64_70	16	1188	1.52	1.68	2.02	1.52	4.30	0.39
64_75	16	1236	1.46	2.10	2.43	1.94	4.96	0.34
64_80	16	1284	0.47	2.34	3.74	1.87	4.69	0.35

Table A.3 (continued)

2 Extra hotels 3 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
66_40	16	575	0.87	0.87	0.87	0.87	0.84	0.08
66_45	16	650	0.77	0.77	0.77	0.77	0.72	0.09
66_50	16	730	2.05	4.34	5.48	2.05	1.02	0.15
66_55	16	825	0.00	1.62	2.42	2.42	1.40	0.16
66_60	16	915	0.55	1.64	2.73	0.55	1.59	0.19
66_125	16	1670	0.90	1.40	1.80	0.00	4.95	0.37
66_130	16	1680	0.30	0.30	0.30	0.89	4.84	0.30
100_30	1	173	0.00	0.00	0.00	0.00	0.34	0.02
100_35	2	241	0.00	0.00	0.00	0.00	0.48	0.05
100_40	2	299	0.00	0.00	0.00	0.00	0.53	0.04
100_45	2	367	0.00	0.00	0.00	0.00	0.71	0.07
102_50	12	181	0.00	0.00	0.00	0.00	0.15	0.02
102_60	12	243	0.00	0.00	0.00	0.00	0.33	0.05
Average	14.14	–	0.43	0.88	1.29	0.93	1.43	0.14
Max	16	–	3.33	4.34	5.48	5.19	4.96	0.44

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.4

3 Extra hotels – 4 trips.

3 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	107	240	0.00	0.00	0.00	0.00	0.28	0.12
T1_70	107	260	0.00	0.00	0.00	0.00	0.31	0.12
T1_73	107	265	0.00	0.00	0.00	0.00	0.32	0.12
T1_75	107	270	0.00	0.00	0.00	0.00	0.35	0.12
T1_80	125	280	0.00	0.00	0.00	1.79	0.36	0.17
T1_85	125	285	0.00	0.00	0.00	0.00	0.37	0.15
T3_65	100	610	0.00	0.00	0.00	0.00	0.33	0.12
T3_75	125	670	0.00	0.00	0.00	2.99	0.34	0.16
T3_80	107	710	0.00	0.00	0.00	0.00	0.31	0.14
T3_85	107	740	0.00	0.00	0.00	0.00	0.34	0.14
T3_90	100	770	0.00	0.00	0.00	0.00	0.37	0.21
T3_95	100	790	0.00	0.00	0.00	0.00	0.44	0.15
T3_100	125	800	0.00	0.00	0.00	5.00	0.38	0.19
T3_105	125	800	0.00	0.42	1.25	0.00	0.41	0.20
64_45	44	816	0.00	0.00	0.00	0.00	1.33	0.18
64_50	72	900	4.67	4.89	5.33	3.33	1.64	0.61
64_55	100	984	0.61	0.61	0.61	0.61	1.56	0.94
64_60	100	1062	0.56	1.88	3.39	2.26	2.09	0.75
64_65	125	1116	0.00	0.36	1.08	1.08	2.51	1.04
64_70	125	1188	1.52	2.19	3.03	1.52	2.88	1.71
64_75	125	1236	2.43	2.91	3.88	2.91	2.78	1.24
64_80	125	1284	0.47	0.93	1.40	1.40	3.71	1.39
66_40	107	575	0.87	0.87	0.87	0.87	0.46	0.16
66_45	88	650	0.77	0.77	0.77	0.77	0.74	0.19
66_50	107	730	2.05	2.05	2.05	2.05	0.83	0.31
66_55	107	825	0.00	0.00	0.00	2.42	0.92	0.37
66_60	107	915	0.55	0.55	0.55	0.55	1.11	0.44
66_125	125	1670	1.20	1.70	2.69	2.10	4.47	1.43
66_130	125	1680	0.30	0.89	1.79	0.60	4.16	1.39
100_30	2	173	0.00	0.00	0.00	0.00	0.36	0.03
100_35	1	241	0.00	0.00	0.00	0.00	0.29	0.03
100_40	1	299	0.00	0.00	0.00	0.00	0.52	0.04
100_45	2	367	0.00	0.00	0.00	0.00	0.50	0.05
102_50	33	181	0.00	0.00	0.00	0.00	0.15	0.03
102_60	60	243	0.00	0.00	0.00	0.00	0.34	0.05
Average	92.80	–	0.46	0.60	0.82	0.92	1.09	0.41
Max	125	–	4.67	4.89	5.33	5.00	4.47	1.71

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.5

5 Extra hotels – 3 trips.

5 Extra hotels 3 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	49	240	0.00	0.00	0.00	0.00	0.07	0.07
T1_70	49	260	0.00	0.64	1.92	0.00	0.07	0.07
T1_73	49	265	0.00	0.00	0.00	0.00	0.07	0.07
T1_75	49	270	0.00	0.00	0.00	0.00	0.10	0.10
T1_80	49	280	0.00	0.00	0.00	0.00	0.07	0.07
T1_85	49	285	0.00	0.00	0.00	0.00	0.08	0.08
T3_65	49	610	0.00	0.00	0.00	0.00	0.09	0.09
T3_75	49	670	0.00	2.49	4.48	2.99	0.08	0.08
T3_80	49	710	0.00	0.00	0.00	0.00	0.08	0.08
T3_85	49	740	0.00	2.70	4.05	4.05	0.08	0.08
T3_90	49	770	0.00	0.00	0.00	3.90	0.08	0.08
T3_95	49	790	0.00	0.00	0.00	0.00	0.09	0.09
T3_100	49	800	0.00	0.00	0.00	0.00	0.08	0.08
T3_105	49	800	0.00	0.00	0.00	0.00	0.08	0.08
64_45	49	816	0.00	0.98	1.47	0.00	0.26	0.26
64_50	49	900	2.67	2.89	3.33	3.33	0.37	0.37
64_55	49	984	2.44	3.05	4.27	3.05	0.42	0.42
64_60	49	1062	0.00	0.75	2.26	0.56	0.48	0.48
64_65	49	1116	0.00	1.08	3.23	2.15	0.54	0.54
64_70	49	1188	1.52	1.85	2.02	1.52	0.57	0.57
64_75	49	1236	0.97	1.62	1.94	1.46	0.62	0.62
64_80	49	1284	0.47	1.40	1.87	1.87	0.58	0.58
66_40	45	575	0.87	0.87	0.87	0.87	0.11	0.11
66_45	47	650	0.77	0.77	0.77	0.77	0.16	0.16
66_50	49	730	2.05	3.42	4.11	2.05	0.22	0.22
66_55	49	825	0.00	0.00	0.00	0.00	0.40	0.40
66_60	49	915	1.64	2.37	2.73	2.73	0.30	0.30
66_125	49	1670	0.60	0.80	0.90	0.90	0.53	0.53
66_130	49	1680	0.30	0.50	0.60	0.30	0.54	0.54
100_30	1	173	0.00	0.00	0.00	0.00	0.02	0.02
100_35	2	241	0.00	0.00	0.00	0.00	0.05	0.05
100_40	2	299	0.00	0.00	0.00	0.00	0.05	0.05
100_45	2	367	0.00	0.00	0.00	0.00	0.07	0.07
102_50	14	181	0.00	0.00	0.00	0.00	0.02	0.02
102_60	17	243	0.00	0.00	0.00	0.00	0.05	0.05
Average	41.51	–	0.41	0.81	1.17	0.93	0.21	0.21
Max	49	–	2.67	3.42	4.48	4.05	0.62	0.62

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.6

6 Extra hotels – 4 trips.

6 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
T1_65	482	240	0.00	0.00	0.00	0.00	0.27	0.27
T1_70	482	260	0.00	0.00	0.00	0.00	0.28	0.28
T1_73	482	265	0.00	1.26	1.89	0.00	0.35	0.35
T1_75	482	270	0.00	0.00	0.00	0.00	0.32	0.32
T1_80	512	280	0.00	1.19	1.79	0.00	0.35	0.35
T1_85	512	285	0.00	0.00	0.00	0.00	0.33	0.33
T3_65	448	610	0.00	0.00	0.00	0.00	0.31	0.31
T3_75	512	670	0.00	0.00	0.00	2.99	0.34	0.34
T3_80	482	710	0.00	0.00	0.00	0.00	0.34	0.34
T3_85	482	740	0.00	0.00	0.00	0.00	0.36	0.36
T3_90	448	770	0.00	0.00	0.00	5.19	0.39	0.39
T3_95	448	790	0.00	0.00	0.00	5.06	0.39	0.39
T3_100	512	800	0.00	0.00	0.00	5.00	0.38	0.38
T3_105	512	800	0.00	0.42	1.25	0.00	0.40	0.40
64_45	252	816	0.00	0.98	2.94	2.94	1.08	1.08
64_50	378	900	3.33	4.22	4.67	3.33	1.47	1.47
64_55	448	984	0.61	1.22	2.44	1.22	1.73	1.73
64_60	448	1062	0.56	0.56	0.56	1.69	2.03	2.03
64_65	512	1116	0.00	0.00	0.00	0.00	2.35	2.35
64_70	512	1188	2.02	2.69	3.03	2.02	2.33	2.33
64_75	512	1236	0.97	1.62	1.94	2.91	2.48	2.48
64_80	512	1284	0.47	1.09	1.87	1.40	2.63	2.63

Table A.6 (continued)

6 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
66_40	302	575	0.87	0.87	0.87	0.87	0.41	0.41
66_45	228	650	0.77	0.77	0.77	0.77	0.54	0.54
66_50	357	730	2.05	2.05	2.05	2.05	0.77	0.77
66_55	400	825	0.00	0.81	2.42	2.42	0.87	0.87
66_60	424	915	0.55	0.55	0.55	0.55	1.10	1.10
66_125	512	1670	0.90	1.40	2.10	1.50	3.18	3.18
66_130	512	1680	0.60	0.79	0.89	0.60	2.92	2.92
100_30	2	173	0.00	0.00	0.00	0.00	0.03	0.03
100_35	1	241	0.00	0.00	0.00	0.00	0.03	0.03
100_40	1	299	0.00	0.00	0.00	0.00	0.04	0.04
100_45	2	367	0.00	0.00	0.00	0.00	0.05	0.05
102_50	60	181	0.00	0.00	0.00	0.00	0.04	0.04
102_60	105	243	0.00	0.00	0.00	0.00	0.09	0.09
Average	379.31	–	0.39	0.64	0.92	1.21	0.88	0.88
Max	512	–	3.33	4.22	4.67	5.19	3.18	3.18

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.7

10 Extra hotels – 4 trips.

10 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	1728	1236	1.46	2.27	2.91	0.97	3.20	2.37
64_80	1728	1284	0.00	0.31	0.47	0.47	3.40	2.63
66_125	1728	1670	0.60	0.60	0.60	0.00	4.85	3.23
66_130	1728	1680	0.30	0.30	0.30	0.30	4.53	3.07
100_50	39	412	0.97	0.97	0.97	0.97	0.99	0.20
100_60	161	504	0.00	0.00	0.00	0.00	1.33	0.78
100_70	276	590	0.00	0.00	0.00	2.54	2.13	1.76
100_80	892	652	0.00	0.51	0.77	1.69	2.99	2.39
100_90	1220	725	0.00	0.00	0.00	2.62	3.60	2.95
100_100	1296	782	1.02	2.30	3.45	2.05	4.53	3.56
100_110	1728	835	2.16	3.79	5.15	0.00	4.86	4.27
100_120	1728	894	0.89	2.68	3.69	0.89	5.58	4.91
100_130	1551	956	0.31	1.99	5.33	4.92	6.75	5.61
100_140	1551	1013	0.00	1.32	3.95	5.82	7.52	6.12
100_150	1728	1057	1.99	1.99	1.99	1.42	8.53	7.17
100_160	1728	1114	0.00	2.99	5.03	8.17	9.52	7.83
100_170	1728	1164	2.32	4.01	4.90	7.47	10.45	7.88
100_180	1728	1201	3.00	3.83	4.83	4.91	10.90	8.88
100_190	1728	1234	1.30	2.00	2.35	4.70	11.52	9.16
100_200	1728	1261	1.74	2.30	2.85	3.25	11.91	9.19
100_210	1728	1284	2.80	3.19	3.74	3.82	12.52	9.25
100_240	1728	1306	0.15	0.41	0.61	0.54	13.30	9.43
Average	1417.18	–	0.96	1.72	2.45	2.61	6.59	5.12
Max	1728	–	3.00	4.01	5.33	8.17	13.30	9.43

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.8

10 Extra hotels – 5 trips.

10 Extra hotels 5 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	17280	1236	1.46	1.62	1.94	2.43	2.86	1.99
64_80	19008	1284	0.47	1.40	2.34	1.87	3.23	2.28
66_125	20736	1670	0.00	0.90	1.80	2.99	4.10	2.76
66_130	20736	1680	0.30	0.30	0.30	1.49	4.33	2.81
100_50	32	412	4.61	4.61	4.61	4.61	0.66	0.15
100_60	167	504	0.00	0.00	0.00	0.00	1.19	0.70
100_70	239	590	0.00	0.73	2.20	0.00	1.66	1.34
100_80	1225	652	0.00	0.00	0.00	0.00	1.96	1.92

(continued on next page)

Table A.8 (continued)

10 Extra hotels 5 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_90	3808	725	0.00	0.00	0.00	0.00	2.68	2.56
100_100	8747	782	1.02	1.02	1.02	2.05	3.64	3.04
100_110	6901	835	0.84	2.28	3.35	3.35	4.65	3.40
100_120	9652	894	0.89	1.98	4.14	6.15	4.91	3.81
100_130	12377	956	1.36	3.80	5.02	8.58	5.07	4.34
100_140	16689	1013	0.00	1.22	3.65	8.88	5.35	4.97
100_150	20736	1057	0.76	2.21	4.16	9.93	6.79	5.57
100_160	19008	1114	4.94	6.40	7.90	7.72	7.37	6.44
100_170	20736	1164	3.87	4.41	5.24	7.65	8.11	6.61
100_180	20736	1201	3.91	4.16	4.33	7.08	8.92	7.19
100_190	20736	1234	0.73	2.59	4.13	5.35	9.78	7.48
100_200	20736	1261	1.27	1.61	1.98	4.68	10.10	7.48
100_210	20736	1284	1.95	1.97	2.02	3.74	10.60	8.17
100_240	20736	1306	0.23	0.56	0.77	2.83	11.46	8.70
Average	13716	–	1.30	1.99	2.77	4.15	5.43	4.26
Max	20736	–	4.94	6.40	7.90	9.93	11.46	8.70

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.9

10 Extra hotels – 6 trips.

10 Extra hotels 6 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	127166	1236	1.94	2.43	2.91	5.83	2.50	1.71
64_80	145152	1284	0.00	0.78	1.40	7.94	2.83	1.88
66_125	248832	1670	1.20	1.50	1.80	2.99	3.62	2.91
66_130	248832	1680	1.19	1.39	1.49	2.98	3.80	3.00
100_50	66	412	0.00	0.00	0.00	0.00	0.62	0.19
100_60	99	504	0.00	0.00	0.00	0.00	0.93	0.36
100_70	1207	590	0.00	0.00	0.00	2.20	1.80	1.41
100_80	540	652	0.00	1.74	2.61	0.00	1.76	1.72
100_90	5924	725	0.00	0.00	0.00	0.00	2.03	1.94
100_100	11960	782	1.66	1.66	1.66	3.45	3.03	2.60
100_110	16951	835	0.00	2.48	7.43	10.90	3.23	3.14
100_120	73516	894	5.15	5.56	6.38	5.82	4.07	3.66
100_130	67239	956	1.36	1.36	1.36	11.09	4.64	3.97
100_140	107503	1013	0.00	1.18	3.55	7.31	4.97	4.27
100_150	129112	1057	0.76	2.27	5.30	12.77	5.52	4.79
100_160	191700	1114	0.00	4.10	9.43	10.23	6.38	5.14
100_170	228096	1164	6.87	7.70	8.68	11.68	7.02	5.70
100_180	228096	1201	1.00	4.00	6.66	12.41	7.59	6.54
100_190	228096	1234	2.11	5.19	8.02	11.43	8.51	6.59
100_200	248832	1261	2.93	4.04	5.71	6.90	8.66	6.71
100_210	248832	1284	0.78	2.26	3.43	7.09	8.78	7.82
100_240	248832	1306	0.23	1.00	1.91	4.82	10.23	8.29
Average	127571.95	–	1.24	2.30	3.62	6.27	4.66	3.83
Max	248832	–	6.87	7.70	9.43	12.77	10.23	8.29

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.10

12 Extra hotels – 4 trips.

12 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	2744	1236	0.97	1.13	1.46	1.46	3.20	2.50
64_80	2744	1284	0.00	0.00	0.00	0.93	3.36	2.63
66_125	2744	1670	0.30	0.50	0.60	0.90	4.54	3.13
66_130	2744	1680	0.30	0.30	0.30	0.60	4.78	2.99
100_50	155	412	0.97	0.97	0.97	0.97	0.90	0.46
100_60	302	504	0.00	0.00	0.00	0.00	1.25	1.17
100_70	530	590	0.51	0.51	0.51	2.54	2.14	2.02
100_80	981	652	0.00	0.00	0.00	0.00	2.82	2.55

Table A.10 (continued)

12 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_90	1812	725	0.00	0.00	0.00	3.03	3.50	2.94
100_100	1733	782	2.69	2.77	2.94	1.02	4.58	3.72
100_110	2744	835	0.84	2.00	3.23	2.51	5.00	4.17
100_120	2744	894	0.89	2.54	4.36	4.36	5.88	4.73
100_130	2548	956	0.31	0.87	1.99	5.44	6.72	5.52
100_140	2548	1013	0.00	0.00	0.00	5.73	7.07	6.08
100_150	2744	1057	3.12	3.47	3.88	1.42	8.14	6.84
100_160	2744	1114	0.00	3.86	5.92	5.30	9.24	7.36
100_170	2744	1164	4.55	5.15	5.84	4.90	10.07	7.78
100_180	2744	1201	3.75	5.16	6.08	6.08	10.49	8.14
100_190	2744	1234	3.08	3.75	4.54	5.43	11.36	8.77
100_200	2744	1261	2.38	2.91	3.25	4.04	11.76	9.08
100_210	2744	1284	2.18	2.65	3.12	2.88	12.18	8.77
100_240	2744	1306	0.46	0.54	0.61	0.61	13.92	8.90
Average	2228.41	–	1.24	1.78	2.25	2.73	6.50	5.01
Max	2744	–	4.55	5.16	6.08	6.08	13.92	9.08

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.11

12 Extra hotels – 5 trips.

12 Extra hotels 5 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	32928	1236	1.46	1.62	1.94	1.94	2.88	2.01
64_80	32928	1284	0.00	0.93	2.34	1.87	3.44	2.34
66_125	38416	1670	0.90	0.90	0.90	1.50	4.23	2.96
66_130	38416	1680	0.30	0.50	0.89	0.60	4.13	2.88
100_50	26	412	4.61	4.61	4.61	4.61	0.55	0.10
100_60	442	504	0.00	0.00	0.00	0.00	1.23	1.00
100_70	1273	590	0.00	0.00	0.00	0.00	1.70	1.46
100_80	3053	652	0.00	0.00	0.00	0.00	2.05	1.82
100_90	5240	725	0.00	0.00	0.00	0.00	2.58	2.49
100_100	15272	782	1.02	1.02	1.02	2.05	3.64	2.90
100_110	10991	835	3.47	3.63	3.95	3.35	4.60	3.47
100_120	15423	894	0.89	0.89	0.89	6.94	4.87	4.07
100_130	24210	956	1.36	2.82	5.75	6.17	4.86	4.47
100_140	27440	1013	0.00	0.43	1.28	5.82	5.39	5.00
100_150	38416	1057	0.76	2.87	4.26	8.23	6.39	5.57
100_160	35672	1114	1.62	3.56	4.58	6.28	7.53	6.37
100_170	38416	1164	3.52	5.15	6.27	8.68	8.35	6.24
100_180	38416	1201	3.33	4.52	6.74	6.99	8.91	6.80
100_190	38416	1234	2.67	3.27	3.57	5.19	9.63	7.16
100_200	38416	1261	2.46	2.83	3.17	2.85	10.07	7.14
100_210	38416	1284	2.26	2.99	3.89	3.27	11.17	7.73
100_240	38416	1306	0.92	1.05	1.30	2.37	11.53	8.73
Average	25029.18	–	1.43	1.98	2.61	3.58	5.44	4.21
Max	38416	–	4.61	5.15	6.74	8.68	11.53	8.73

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.12

12 Extra hotels – 6 trips.

12 Extra hotels 6 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	275971	1236	2.91	3.56	3.88	3.40	1.85	2.00
64_80	268912	1284	0.47	0.93	1.40	3.27	2.21	2.07
66_125	537824	1670	0.30	1.10	1.50	2.40	2.83	3.04
66_130	537824	1680	0.30	0.50	0.60	0.89	2.84	3.04
100_50	246	412	0.00	0.00	0.00	0.00	0.45	0.54
100_60	252	504	0.00	0.00	0.00	0.00	0.70	0.78
100_70	1819	590	0.00	0.00	0.00	2.20	1.27	1.33

(continued on next page)

Table A.12 (continued)

12 Extra hotels 6 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_80	3203	652	0.00	0.00	0.00	2.91	1.41	1.74
100_90	13194	725	0.00	1.89	5.66	0.00	1.52	2.09
100_100	27365	782	1.66	1.66	1.66	4.86	2.14	2.51
100_110	23330	835	4.55	4.71	4.91	7.78	2.71	3.14
100_120	88540	894	3.47	3.47	3.47	3.80	3.11	3.50
100_130	105633	956	1.36	2.89	5.96	6.49	3.56	3.95
100_140	187852	1013	0.00	0.00	0.00	9.08	3.72	4.08
100_150	215705	1057	0.76	1.86	4.07	9.37	4.18	5.00
100_160	358706	1114	5.21	6.16	7.00	10.05	4.89	5.09
100_170	412832	1164	4.64	6.24	8.68	11.86	5.12	5.43
100_180	422576	1201	0.25	3.05	5.58	10.16	5.61	6.86
100_190	460992	1234	3.89	4.19	4.78	9.48	5.94	6.39
100_200	537824	1261	2.38	3.67	4.36	12.85	6.30	6.60
100_210	537824	1284	2.88	2.99	3.04	9.74	6.82	7.24
100_240	537824	1306	0.77	1.20	1.68	3.29	7.50	8.02
Average	252556.73	–	1.63	2.28	3.10	5.63	3.49	3.84
Max	537824	–	5.21	6.24	8.68	12.85	7.50	8.02

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.13
15 Extra hotels – 4 trips.

15 Extra hotels 4 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	4913	1236	0.97	1.29	1.46	0.97	3.29	2.53
64_80	4913	1284	0.00	0.00	0.00	0.47	3.69	2.59
66_125	4913	1670	0.30	0.90	1.20	0.90	5.00	3.16
66_130	4913	1680	0.00	0.30	0.60	0.30	4.73	3.07
100_50	37	412	0.97	0.97	0.97	0.97	0.99	0.19
100_60	145	504	0.00	0.00	0.00	0.00	1.38	0.75
100_70	311	590	0.51	0.51	0.51	2.54	2.15	1.67
100_80	1036	652	0.00	0.00	0.00	0.00	3.15	2.51
100_90	2545	725	0.00	1.89	3.03	2.21	3.51	3.07
100_100	1951	782	1.02	1.45	2.30	1.92	4.34	3.77
100_110	4533	835	3.11	3.15	3.23	4.19	5.28	4.18
100_120	4847	894	1.68	3.09	3.91	3.69	5.86	4.76
100_130	4576	956	1.88	2.72	4.08	4.39	6.33	5.29
100_140	4624	1013	0.99	2.86	4.94	4.74	7.70	6.05
100_150	4913	1057	1.99	2.11	2.37	1.42	8.40	6.27
100_160	4913	1114	4.13	5.15	5.75	4.76	8.65	7.02
100_170	4913	1164	4.12	4.61	4.90	5.76	9.27	7.69
100_180	4913	1201	1.25	2.61	3.41	5.00	10.59	8.82
100_190	4913	1234	3.16	3.94	4.94	5.43	11.64	8.62
100_200	4913	1261	1.43	2.09	2.70	4.76	12.29	9.06
100_210	4913	1284	1.48	2.44	3.58	3.50	13.03	8.91
100_240	4913	1306	0.15	0.23	0.38	0.69	14.60	8.84
Average	3798.33	–	1.32	1.92	2.47	2.66	6.63	4.95
Max	4913	–	4.13	5.15	5.75	5.76	14.60	9.06

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.14
15 Extra hotels – 5 trips.

15 Extra hotels 5 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	73695	1236	1.46	1.78	1.94	1.46	2.80	2.07
64_80	73695	1284	0.93	1.09	1.40	1.87	3.12	2.27
66_125	83521	1670	0.90	1.10	1.50	3.29	4.40	2.85
66_130	83521	1680	0.60	0.60	0.60	1.49	4.21	2.98
100_50	13	412	4.61	4.61	4.61	4.61	0.57	0.10
100_60	260	504	0.00	0.00	0.00	0.00	1.09	0.97
100_70	297	590	0.00	0.73	2.20	0.00	1.68	1.36
100_80	1459	652	0.00	0.00	0.00	0.00	2.14	2.11
100_90	3524	725	0.00	0.00	0.00	0.00	2.68	2.72

Table A.14 (continued)

15 Extra hotels 5 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_100	15258	782	1.02	2.00	3.96	2.05	3.58	3.02
100_110	13318	835	0.00	0.96	2.87	8.02	4.52	3.27
100_120	16903	894	0.89	2.61	3.47	11.52	4.99	3.60
100_130	37279	956	1.36	2.89	5.96	7.01	5.05	4.10
100_140	66840	1013	0.00	1.45	4.34	7.01	5.46	4.63
100_150	81857	1057	0.76	3.69	5.58	7.95	6.74	5.28
100_160	78608	1114	3.77	5.80	7.18	7.99	7.44	5.92
100_170	83521	1164	1.89	4.07	5.15	5.76	8.03	6.65
100_180	83521	1201	2.16	3.66	4.91	5.50	9.01	6.77
100_190	83521	1234	3.97	4.16	4.29	6.56	9.30	7.07
100_200	83521	1261	3.41	3.73	4.36	6.42	9.99	7.78
100_210	83521	1284	2.96	3.25	3.43	5.84	10.42	8.23
100_240	83521	1306	0.00	0.64	1.00	2.99	11.74	8.59
Average	51417.00	–	1.40	2.22	3.13	4.42	5.41	4.20
Max	83521	–	4.61	5.80	7.18	11.52	11.74	8.59

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.15

15 Extra hotels – 6 trips.

15 Extra hotels 6 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
64_75	845770	1236	2.43	2.59	2.91	4.85	2.43	2.59
64_80	751689	1284	0.00	0.62	1.40	0.93	2.86	2.53
66_125	1419857	1670	0.00	0.30	0.60	2.69	3.90	4.08
66_130	1419857	1680	0.30	0.40	0.60	1.49	3.86	4.21
100_50	116	412	0.00	0.00	0.00	0.00	0.68	0.35
100_60	131	504	0.00	0.00	0.00	0.00	0.88	0.47
100_70	2824	590	0.00	0.00	0.00	1.02	1.69	1.45
100_80	2214	652	0.00	0.87	2.61	0.00	1.90	1.91
100_90	14640	725	0.00	0.00	0.00	0.00	2.40	2.35
100_100	27171	782	1.66	2.51	4.22	11.25	2.94	2.68
100_110	44708	835	5.75	6.11	6.47	9.10	3.73	2.81
100_120	193555	894	5.15	5.44	5.93	10.63	4.48	3.46
100_130	181798	956	1.36	4.95	6.80	11.40	4.89	4.14
100_140	277966	1013	0.00	4.61	10.46	13.52	4.80	3.96
100_150	318708	1057	0.76	1.83	3.97	11.64	5.66	4.62
100_160	1050669	1114	2.51	4.73	8.89	9.87	6.50	5.38
100_170	1177116	1164	1.12	5.41	8.16	12.71	7.00	5.83
100_180	1242989	1201	0.83	4.11	7.74	11.66	7.50	7.38
100_190	1252815	1234	3.24	4.32	5.92	10.70	8.26	7.19
100_200	1419857	1261	1.90	2.80	3.65	12.85	8.88	7.99
100_210	1419857	1284	2.57	3.40	4.36	10.59	9.29	8.58
100_240	1419857	1306	0.92	1.07	1.15	3.91	10.69	10.38
Average	658371.09	–	1.39	2.55	3.90	6.86	4.78	4.29
Max	1419857	–	5.75	6.11	10.46	13.52	10.69	10.38

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.16

15 Extra hotels – 8 trips.

15 Extra hotels 8 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_100	68358	782	2.43	2.43	2.43	3.45	2.45	2.35
100_110	663646	835	0.84	0.84	0.84	12.69	3.28	3.13
100_120	423048	894	1.90	1.90	1.90	22.37	3.66	2.87
100_130	1321340	956	1.36	1.36	1.36	17.57	3.66	4.61
100_140	3486322	1013	0.00	0.00	0.00	18.85	4.13	8.43
100_150	9032502	1057	1.23	1.23	1.23	17.22	4.23	17.38
100_160	18397252	1114	8.80	9.75	11.67	16.61	5.01	336.61
100_170	–	1164	1.98	6.64	9.02	–	5.47	–
100_180	–	1201	5.16	5.80	6.16	–	6.22	–

(continued on next page)

Table A.16 (continued)

15 Extra hotels 8 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_190	–	1234	4.13	5.78	7.29	–	6.15	–
100_200	–	1261	4.20	5.21	5.71	–	7.05	–
100_210	–	1284	4.52	4.72	4.91	–	7.22	–
100_240	–	1306	1.84	1.97	2.14	–	8.58	–
Average	–	–	2.95	3.66	4.20	15.54	5.16	53.62
Max	410338673	–	8.80	9.75	11.67	22.37	8.58	336.61

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.17
15 Extra hotels – 10 trips.

15 Extra hotels 10 Trips								
Instances			MA-AVG-Gap(%)			SVNS-AVG-Gap(%)	MA-CPU(s)	SVNS-CPU(s)
Name	TNFS	OPT	Best	Average	Worst	Gap	Avg CPU	CPU
100_140	–	1013	0.00	0.00	0.00	–	3.31	–
100_150	–	1057	1.80	1.80	1.80	–	3.71	–
100_160	–	1114	9.16	9.25	9.43	–	4.09	–
100_170	–	1164	0.00	3.04	9.11	–	4.53	–
100_180	–	1201	7.41	8.49	9.74	–	5.30	–
100_190	–	1234	6.73	8.48	10.37	–	5.24	–
100_200	–	1261	6.66	7.14	7.93	–	5.60	–
100_210	–	1284	0.93	5.22	7.63	–	6.21	–
100_240	–	1306	1.30	1.89	2.22	–	7.41	–
Average	–	–	3.78	5.03	6.47	–	5.04	–
Max	1.185878765 × 10 ¹¹	–	9.16	9.25	10.37	–	7.41	–

Bold numbers indicate finding the optimal solution by 0.00% gap.

Table A.18
Set4.

SET 4												
Instances		Best known results			Gap(%) w BF		Gap(%) w UB		Gap(%) w OV		CPU(s)	
Name	TNFS	BF	UB	Optimal Value	MA	SVNS	MA	SVNS	MA	SVNS	MA	SVNS
100_20_2	4	–	–	247	–	–	–	–	0.00	0.00	0.51	0.25
100_25_2	5	–	–	385	–	–	–	–	0.00	0.00	1.90	0.31
102_35_2	2	–	–	157	–	–	–	–	3.82	3.82	0.37	0.06
102_40_2	2	–	–	210	–	–	–	–	0.00	0.00	0.56	0.10
102_45_2	2	–	–	266	–	–	–	–	0.00	0.00	0.89	0.16
100_20_3	19	357	376	–	–3.08	–3.08	2.13	2.13	–	–	1.46	0.33
100_25_3	25	495	568	–	–5.86	–5.86	7.75	7.75	–	–	2.66	0.57
102_35_3	7	230	380	–	–40.87	–40.87	14.74	14.74	–	–	1.05	0.13
102_40_3	7	299	493	–	–29.10	–28.09	21.70	22.31	–	–	1.10	0.22
102_45_3	7	356	579	–	–24.16	–24.16	23.66	23.66	–	–	1.21	0.28
Average	8				–20.61	–20.41	14.12	14.12	0.76	0.76	1.17	0.24
Max	25				–3.80	–3.80	23.66	23.66	3.82	3.82	3.19	0.57

Bold numbers indicate finding the optimal solution by 0.00% gap.

References

- Angelelli, E., & Speranza, M. G. (2002a). The periodic vehicle routing problem with intermediate facilities. *European Journal of Operational Research*, 137, 233–247.
- Angelelli, E., & Speranza, M. G. (2002b). The application of a vehicle routing model to a waste-collection problem: two case studies. *Journal of the Operational Research Society*, 53, 944–952.
- Beasley, J. E. (1983). Route first–Cluster second methods for vehicle routing. *Omega*, 11, 403–408.
- Benjamin, A. M., & Beasley, J. E. (2010). Metaheuristics for the waste collection vehicle routing problem with time windows, driver rest period and multiple disposal facilities. *Computers & Operations Research*, 37, 2270–2280.
- Boudia, M., & Prins, C. (2009). A memetic algorithm with dynamic population management for an integrated production–Distribution problem. *European Journal of Operational Research*, 195, 703–715.
- Bouly, H., Dang, D.-C., Moukrim, A., (2010). A memetic algorithm for the team orienteering problem. *4OR*, 8, 49–70.
- Castro, M., Sörensen, K., Vansteenwegen, P., & Goos, P. (2013). A memetic algorithm for the travelling salesperson problem with hotel selection. *Computers & Operations Research*, 40, 1716–1728.
- Crevier, B., Cordeau, J.-F., & Laporte, G. (2007). The multi-depot vehicle routing problem with inter-depot routes. *European Journal of Operational Research*, 176, 756–773.
- Dang, D.-C., Guibadj, R. N., & Moukrim, A. (2013). An effective PSO-inspired algorithm for the team orienteering problem. *European Journal of Operational Research*, 229, 332–344.

- Divsalar, A., Vansteenwegen, P., & Cattrysse, D. (2013). A variable neighborhood search method for the orienteering problem with hotel selection. *International Journal of Production Economics*, 145, 150–160.
- Gendreau, M., Laporte, G., & Semet, F. (1998). A branch and cut algorithm for the undirected selective traveling salesman problem. *Networks*, 32, 263–273.
- Ghiani, G., & Federico, N. (2001). The capacitated arc routing problem with intermediate facilities. *Networks*, 37, 134–143.
- Ghiani, G., Guerriero, F., Laporte, G., & Musmanno, R. (2004). Tabu search heuristics for the arc routing problem with intermediate facilities under capacity and length restrictions. *Journal of Mathematical Modelling and Algorithms*, 3, 209–223.
- Ghiani, G., Laganà, D., Laporte, G., & Mari, F. (2008). Ant colony optimization for the arc routing problem with intermediate facilities under capacity and length restrictions. *Journal of Heuristics*, 16, 211–233.
- Golden, B. L., Levy, L., & Vohra, R. (1987). The orienteering problem. *Naval Research Logistics*, 34, 307–318.
- Kek, A. G. H., Cheu, R. L., & Meng, Q. (2008). Distance-constrained capacitated vehicle routing problems with flexible assignment of start and end depots. *Mathematical and Computer Modelling*, 47, 140–152.
- Kim, B.-I., Kim, S., & Sahoo, S. (2006). Waste collection vehicle routing problem with time windows. *Computers & Operations Research*, 33, 3624–3642.
- Mendoza, J. E., Castanier, B., Guéret, C., Medaglia, A. L., & Velasco, N. (2010). A memetic algorithm for the multi-compartment vehicle routing problem with stochastic demands. *Computers & Operations Research*, 37, 1886–1898.
- Moscato, P. (1999). Memetic algorithms: A short introduction. In D. Corne, M. Dorigo, & F. Glover (Eds.), *New Ideas in Optimization* (pp. 219–234). McGraw-Hill.
- Ngueveu, S. U., Prins, C., & Wolfler Calvo, R. (2010). An effective memetic algorithm for the cumulative capacitated vehicle routing problem. *Computers & Operations Research*, 37, 1877–1885.
- Potvin, J.-Y. (2009). State-of-the art review—evolutionary algorithms for vehicle routing. (F.B. Pereira & J. Tavares, (Eds.)) *INFORMS Journal on Computing* 21, 518–548.
- Prins, C. (2004). A simple and effective evolutionary algorithm for the vehicle routing problem. *Computers & Operations Research*, 31, 1985–2002.
- Schilde, M., Doerner, K. F., Hartl, R. F., & Kiechle, G. (2009). Metaheuristics for the bi-objective orienteering problem. *Swarm Intelligence*, 3, 179–201.
- Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35, 254–265.
- Sörensen, K., & Sevaux, M. (2006). MAPM: memetic algorithms with population management. *Computers & Operations Research*, 33, 1214–1225.
- Talbi, E.-G. (2009). *Metaheuristics: From design to implementation*. Wiley.
- Tarantilis, C. D., Zachariadis, E. E., & Kiranoudis, C. T. (2008). A hybrid guided local search for the vehicle-routing problem with intermediate replenishment facilities. *INFORMS Journal on Computing*, 20, 154–168.
- Tsiligirides, T. (1984). Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35, 797–809.
- Vansteenwegen, P., Souffriau, W., & Sörensen, K. (2012). The travelling salesperson problem with hotel selection. *Journal of the Operational Research Society*, 63, 207–217.
- Vansteenwegen, P., Souffriau, W., & Van Oudheusden, D. (2011). The orienteering problem: A survey. *European Journal of Operational Research*, 209, 1–11.
- Vansteenwegen, P., Souffriau, W., Vanden Berghe, G., & Van Oudheusden, D. (2011). The City Trip Planner: An expert system for tourists. *Expert Systems with Applications*, 38, 6540–6546.